ABSTRACT

The project focuses on the design and implementation of a 32-bit single-cycle RISC-V processor architecture without pipelining. RISC-V is an open-standard Instruction Set Architecture (ISA) based on the Reduced Instruction Set Computer (RISC) design principle. The primary objective of this project is to understand the internal working of a processor and the execution of instructions through its major functional units.

The designed processor consists of essential components, including the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), Data Memory, and supporting multiplexers. Each instruction is fetched, decoded, and executed within a single clock cycle. The Control Unit generates the necessary control signals based on the instruction, while the ALU performs arithmetic and logical operations. The Register File stores operands and results, and the Data Memory supports load and store instructions. The architecture also handles instruction execution and program flow through appropriate branching and control mechanisms.

Since the processor does not use pipelining, only one instruction is processed at a time, and all stages of instruction execution are completed within one clock cycle. This simplifies the processor design and makes it easier to understand the interaction between different hardware components. However, the clock period must be sufficiently long to accommodate the instruction with the maximum execution delay.

The project aims to demonstrate the fundamental principles of processor design, instruction execution, and datapath and control unit integration. The implementation and simulation of the architecture help verify the correct functioning of various instructions and provide a practical understanding of RISC-V processor organization.

TABLE OF CONTENTS

1. INTRODUCTION 1

2. OBJECTIVES OF THE PROJECT 2

3. CIRCUIT DIAGRAM 3

4. MAIN COMPONENTS 3

5. FLOWCHART 6

6. OPERATION 7

6.1 Role of the Register File 7

6.2 Interpretation Table 8

7. OUTPUT 9

7.1 Output 1 9

7.2 Output 2 10

8. ADVANTAGES AND DISADVANTAGES 11

8.1 Advantages 11

8.2 Disadvantages 11

9. APPLICATIONS 11

10. CONCLUSION 12

11. RESULT 12

LIST OF FIGURES

Figure 1: Circuit diagram of the single-cycle RISC-V datapath....3
Figure 2: Flowchart of single-cycle instruction execution....6
Figure 3: Initial register file values....7
Figure 4: Instruction memory contents....8
Figure 5: Simulation waveform (Output 1)....9
Figure 6: Simulation waveform (Output 2)....10

LIST OF TABLES

Table 1: Main control unit signal values....3
Table 2: Control unit input and output signals....4
Table 3: ALU control signal generation....4
Table 4: ALU control lines and functions....5
Table 5: RISC-V instruction formats....5
Table 6: Interpretation of assembly instructions....8

1. INTRODUCTION

A processor is the central component of a computer system responsible for executing instructions and performing arithmetic, logical, and data-processing operations. Processor architecture defines how instructions are processed and how different hardware components work together to execute a program efficiently. Understanding processor architecture is essential for studying computer organization, digital systems, and computer hardware design.

RISC-V is an open-standard Instruction Set Architecture (ISA) based on the Reduced Instruction Set Computer (RISC) design philosophy. It provides a simple, modular, and extensible instruction set that supports the development of processors for various applications. Due to its open nature and straightforward instruction format, RISC-V has become an important platform for academic research, processor design, and hardware development.

This project focuses on the design and simulation of a 32-bit single-cycle RISC-V processor without pipelining. In a single-cycle architecture, each instruction is executed completely within one clock cycle. The processor performs instruction fetching, decoding, execution, memory access, and write-back during the same cycle, depending on the type of instruction. Unlike pipelined processors, which overlap the execution stages of multiple instructions, this architecture completes one instruction before proceeding to the next.

The processor consists of several essential functional units, including the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), Data Memory, and multiplexers. The Program Counter stores the address of the current instruction, while Instruction Memory provides the instruction to be executed. The Control Unit decodes the instruction and generates the required control signals. The Register File stores data and provides operands, whereas the ALU performs arithmetic and logical operations. Data Memory is used for load and store instructions, and the write-back unit transfers the final result to the appropriate register.

The primary objective of this project is to understand the working principles of a RISC-V processor and the interaction between its datapath and control unit. Through implementation and simulation, the project demonstrates how instructions are fetched, decoded, and executed, how data flows between functional units, and how the Program Counter is updated to maintain the correct sequence of execution.

Overall, this project provides practical insight into the internal organization of a 32-bit RISC-V processor, instruction execution, datapath design, and control signal generation. It bridges the gap between theoretical concepts of computer architecture and their practical implementation through software-based design and simulation.

2. OBJECTIVES OF THE PROJECT

The main objective of this project is to design, implement, and simulate a 32-bit single-cycle RISC-V processor without pipelining to understand the fundamental concepts of processor architecture and instruction execution.

The specific objectives of the project are as follows:

1. **To design a 32-bit RISC-V processor:** Develop a processor architecture based on the RISC-V Instruction Set Architecture (ISA) that supports the execution of selected instructions.
2. **To understand single-cycle execution:** Demonstrate how an instruction is fetched, decoded, and executed completely within a single clock cycle.
3. **To implement the datapath:** Design and integrate essential functional units, including the Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Data Memory, and multiplexers.
4. **To develop the Control Unit:** Generate appropriate control signals according to the instruction type to coordinate the operation of different processor components.
5. **To implement arithmetic and logical operations:** Enable the processor to perform supported operations such as addition, subtraction, and bitwise logical operations.
6. **To support data transfer instructions:** Implement load and store operations to transfer data between the processor's registers and Data Memory, as supported by the design.
7. **To implement program flow control:** Demonstrate sequential instruction execution and appropriate updates to the Program Counter for supported branching and jump instructions.
8. **To verify processor functionality:** Simulate the design and analyze the outputs to confirm the correct execution of instructions and the proper functioning of individual components.

9. **To understand non-pipelined architecture:** Study the characteristics, advantages, and limitations of a single-cycle processor and understand how it differs from pipelined processor designs.
10. **To establish a foundation for advanced processor design:** Develop practical knowledge of processor organization that can be extended to multicycle and pipelined architectures in future studies.

3. CIRCUIT DIAGRAM

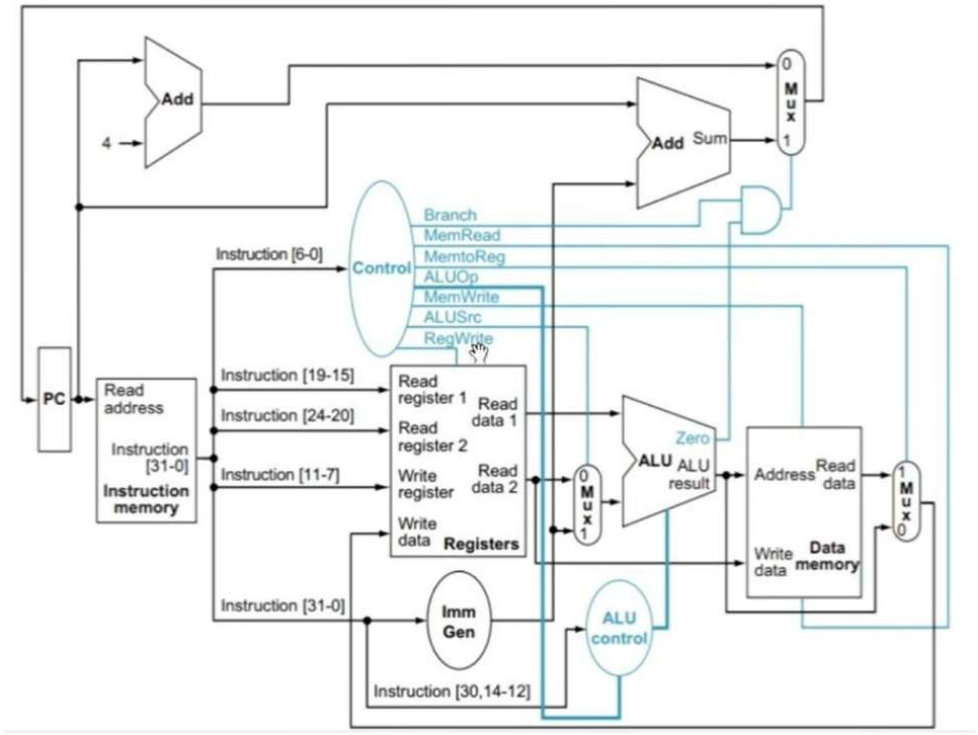


Figure 1: Circuit diagram of the single-cycle RISC-V datapath

4. MAIN COMPONENTS

- **Program Counter (PC):** The Program Counter is a 32-bit register that stores the address of the instruction to be fetched. Normally, it advances by 4 bytes to point to the next instruction. For a taken branch, it is updated with the branch target address.
- **Instruction Memory:** Instruction Memory stores the program instructions. It receives the address from the PC and provides the corresponding instruction to the processor for decoding and execution.

- **Main Control Unit:** The Main Control Unit decodes the opcode of the instruction and generates control signals such as ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, Branch, ALUOp1, and ALUOp0. These signals control the operation of the processor's datapath.

Table 1: Main control unit signal values

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Table 2: Control unit input and output signals

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

- **Register File:** The Register File contains 32 general-purpose registers in the standard RV32I architecture. It reads data from source registers rs1 and rs2 and writes the result to destination register rd when the RegWrite signal is enabled.
- **The Write-Back Unit:** It selects the value that will be written into the destination register. For arithmetic instructions, it selects the ALU result; for load instructions, it selects the data read from Data Memory.
- **Immediate Generator (Imm Gen):** The Immediate Generator extracts immediate values from instruction bits and arranges them according to the relevant instruction format. It sign-extends the immediate to 32 bits when required and supplies it to the ALU or branch-address calculation circuit.

- **ALU Control Unit:** The ALU Control Unit receives ALUOp signals from the Main Control Unit and uses instruction fields such as funct3 and funct7 to generate the ALU's operation-select signal. It determines whether the ALU performs addition, subtraction, AND, OR etc.

Table 3: ALU control signal generation

ALUOp		Funct7 field							Funct3 field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

- **Arithmetic Logic Unit (ALU):** The ALU performs arithmetic and logical operations on its input operands. It also calculates effective memory addresses for load and store instructions and produces comparison information used by branch logic.

Table 4: ALU control lines and functions

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

- **Data Memory:** Data Memory stores the data used by the program. The lw instruction reads a word from memory, while the sw instruction writes a word into memory. The memory address is calculated by the ALU.
- **Multiplexers (MUX):** Multiplexers select one of several inputs according to control signals. They select the ALU's second operand, the data to be written back to the register file, and the next PC value.
- **Adders:** The first adder calculates the address of the next sequential instruction. The second adder calculates the branch target address by adding the PC to the branch offset.
- **Branch Decision Logic:** The branch decision logic uses the Branch control signal and the ALU's comparison result to determine whether the processor should jump to the branch target or continue to the next sequential instruction.

Table 5: RISC-V instruction formats

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

5. FLOWCHART

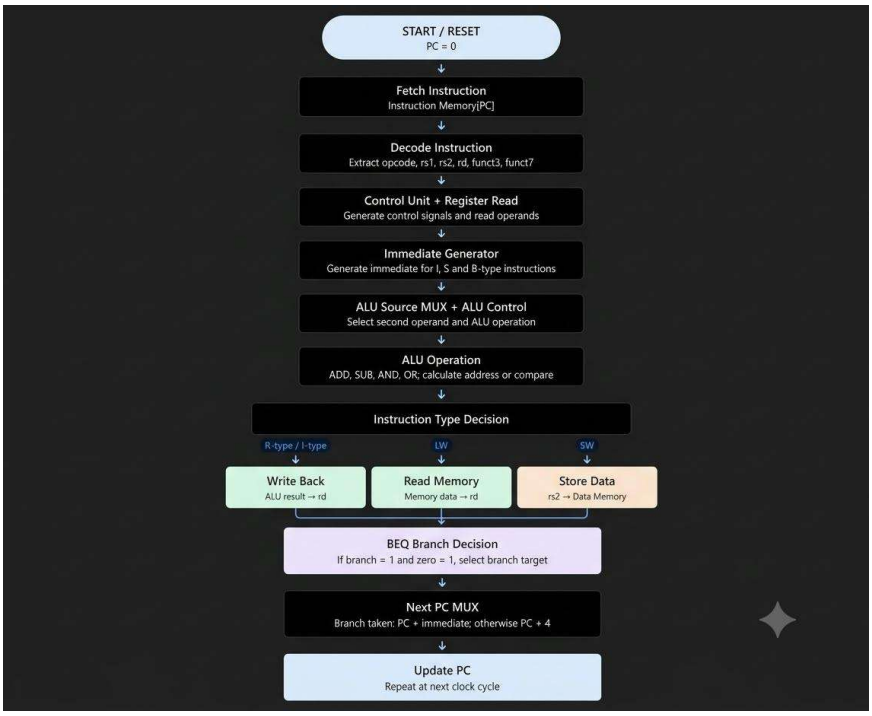


Figure 2: Flowchart of single-cycle instruction execution

The flowchart illustrates the initial stages of instruction execution in a **32-bit single-cycle RISC-V processor without pipelining**. It explains how an instruction is fetched from instruction memory, decoded, and processed through the control unit, immediate generator, and ALU to perform the required operation.

1. **Start / Reset:** The processor initializes, and the Program Counter (PC) is set to 0. This indicates the starting address from which the first instruction is fetched.
2. **Fetch Instruction:** The instruction memory uses the address stored in the PC to fetch the instruction that needs to be executed.

3. **Decode Instruction:** The fetched instruction is decoded to extract the opcode, source register addresses (rs1 and rs2), destination register address (rd), function code (funct3), and function code (funct7). These fields help determine the type of operation to be performed.
4. **Control Unit + Register Read:** The control unit examines the instruction and generates the necessary control signals. Simultaneously, the register file reads the source operands from the registers specified by rs1 and rs2.
5. **Immediate Generator:** This unit generates the immediate value from the instruction for supported I-type, S-type, and B-type instructions. The generated value is used in arithmetic operations, memory address calculations, or branch-related operations.
6. **ALU Source MUX + ALU Control:** The ALU source multiplexer selects the appropriate second operand, either from the register file or from the immediate generator. The ALU control unit determines the specific arithmetic or logical operation to be performed based on the instruction.
7. **ALU Operation:** The Arithmetic Logic Unit performs the selected operation, such as addition (ADD), subtraction (SUB), bitwise AND, or bitwise OR. It can also calculate memory addresses or compare operands for supported branch instructions.

6. OPERATION

```
registers[0]  = 0;
registers[1]  = 4;
registers[2]  = 2;
registers[3]  = 24;
registers[4]  = 4;
registers[5]  = 1;
registers[6]  = 44;
registers[7]  = 4;
registers[8]  = 2;
registers[9]  = 1;
registers[10] = 23;
registers[11] = 4;
registers[12] = 90;
registers[13] = 10;
registers[14] = 20;
registers[15] = 30;
registers[16] = 40;
registers[17] = 50;
registers[18] = 60;
registers[19] = 70;
registers[20] = 80;
registers[21] = 80;
registers[22] = 90;
registers[23] = 90;
registers[24] = 70;
registers[25] = 65;
registers[26] = 4;
registers[27] = 32;
registers[28] = 12;
registers[29] = 34;
registers[30] = 5;
```

Figure 3: Initial register file values

6.1 Role of the Register File

The register file is an important component of the processor datapath. It temporarily stores operands and intermediate results required for instruction execution.

Its main functions are:

- **Operand storage:** Stores the input values needed by arithmetic and logical instructions.
- **Data reading:** Supplies register values to the ALU and address-generation circuitry.
- **Result storage:** Receives results from the ALU or data memory during write-back operations.
- **Instruction support:** Provides operands for load, store, and conditional branch instructions.

In a typical RISC-V register file, two source registers can be read simultaneously, and one destination register can be written during a clock cycle.

```
IMemory[0]  = 32'b00000000000000000000000000000000;  // no operation
IMemory[4]  = 32'b0000000_11001_10000_000_01101_0110011;  // add x13, x16, x25
IMemory[8]  = 32'b0100000_00011_01000_000_00101_0110011;  // sub x5, x8, x3
IMemory[12] = 32'b0000000_00011_00010_111_00001_0110011;  // and x1, x2, x3
IMemory[16] = 32'b0000000_00101_00011_110_00100_0110011;  // or  x4, x3, x5

IMemory[20] = 32'b000000000011_10101_000_10110_0010011;  // addi x22, x21, 3
IMemory[24] = 32'b000000000001_01000_110_01001_0010011;  // ori x9, x8, 1

IMemory[28] = 32'b000000001111_00101_010_01000_0000011;  // lw x8, 15(x5)
IMemory[32] = 32'b000000000011_00011_010_01001_0000011;  // lw x9, 3(x3)

IMemory[36] = 32'b0000000_01111_00101_010_01100_0100011;  // sw x15, 12(x5)
IMemory[40] = 32'b0000000_01110_00110_010_01010_0100011;  // sw x14, 10(x6)

IMemory[44] = 32'h00948663;  // beq x9, x9, 12
```

Figure 4: Instruction memory contents

6.2 Interpretation Table

Table 6: Interpretation of assembly instructions

Assembly Instruction	Operation	Description
NOP (No Operation)	No operation	The processor performs no meaningful computation.
`add x13, x16, x25`	Addition	Adds the values stored in registers `x16` and `x25` and stores the result in `x13`.
`sub x5, x8, x3`	Subtraction	Subtracts the value in `x3` from the value in `x8` and stores the result in `x5`.
`and x1, x2, x3`	Bitwise AND	Performs a bitwise AND operation on registers `x2` and `x3` and stores the result in `x1`.

Assembly Instruction	Operation	Description
`or x4, x3, x5`	Bitwise OR	Performs a bitwise OR operation on registers `x3` and `x5` and stores the result in `x4`.
`addi x22, x21, 3`	Immediate addition	Adds the constant 3 to the value in `x21` and stores the result in `x22`.
`ori x9, x8, 1`	Immediate OR	Performs a bitwise OR between the value in `x8` and the immediate value 1, storing the result in `x9`.
`lw x8, 15(x5)`	Load word	Loads a 32-bit word from data memory at address `x5` + 15 into `x8`.
`lw x9, 3(x3)`	Load word	Loads a 32-bit word from data memory at address `x3` + 3 into `x9`.
`sw x15, 12(x5)`	Store word	Stores the value in `x15` into data memory at address `x5` + 12.
`sw x14, 10(x6)`	Store word	Stores the value in `x14` into data memory at address `x6` + 10.
`beq x9, x9, 12`	Conditional branch	Compares `x9` with itself. Since both operands are equal, the branch is always taken.

7. OUTPUT

7.1 Output 1

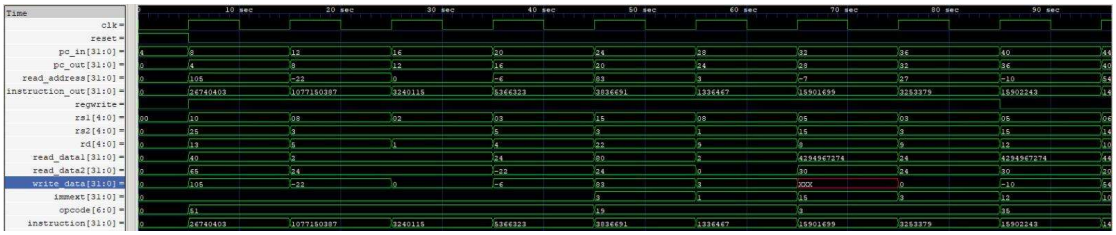


Figure 5: Simulation waveform (Output 1)

1. **ADD Operation:** The processor adds the values of registers x16 and x25. The values are 40 and 65, respectively, producing a result of **105**, which is written to register x13.
2. **SUB Operation:** The processor subtracts the value of register x3 (24) from register x8 (2). The result is **−22**, which is stored in register x5.
3. **AND Operation:** The processor performs a bitwise AND operation on registers x2 (2) and x3 (24). The result is **0**, which is stored in register x1.
4. **OR Operation:** The processor performs a bitwise OR operation on registers x3 (24) and x5 (−22). The expected result is **−6**, which is stored in register x4.

5. **ADDI Operation:** The processor adds the immediate value 3 to the value of register x21 (80). The result is **83**, which is stored in register x22.
6. **ORI Operation:** The processor performs a bitwise OR operation between register x8 (2) and the immediate value 1. The result is **3**, which is stored in register x9.
7. **First Load Word (LW):** The processor calculates the effective memory address by adding the offset 15 to the value of register x5 (−22). The calculated address is **−7**. The data stored at this address is then intended to be loaded into register x8, subject to the memory implementation.
8. **Second Load Word (LW):** The processor adds the offset 3 to the value of register x3 (24), calculating an effective address of **27**. The data at this address is intended to be loaded into register x9.
9. **First Store Word (SW):** The processor adds the offset 12 to the value of register x5 (−22), calculating an effective address of **−10**. The value of register x15 is intended to be stored at this address.
10. **Second Store Word (SW):** The processor adds the offset 10 to the value of register x6 (44), calculating an effective address of **54**. The value of register x14 is intended to be stored at this address.
11. **BEQ Operation:** The processor compares register x9 with itself. Since both operands refer to the same register, the branch condition is true, and the processor should branch to the specified target address if the instruction is correctly encoded and executed.

7.2 Output 2

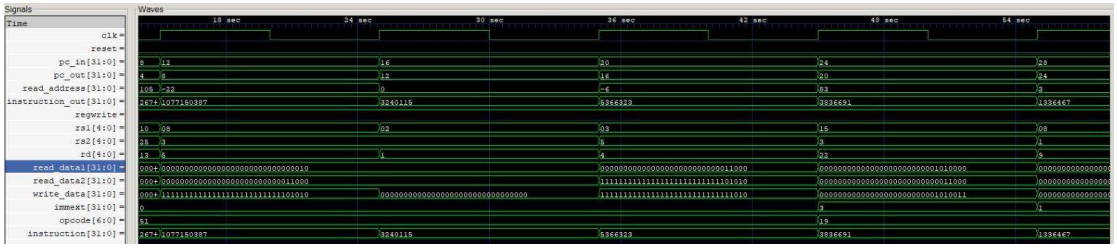


Figure 6: Simulation waveform (Output 2)

1. **SUB Operation:** The processor reads the values 2 and 24 from the source registers and performs subtraction. The result is **−22**, which appears in the datapath.
2. **AND Operation:** The processor performs a bitwise AND operation on the values 2 and 24. Since their binary representations have no common set bits, the result is **0**.

3. **OR Operation:** The processor performs a bitwise OR operation using the values 24 and −22. The expected 32-bit two's-complement result is **−6**.
4. **ADDI Operation:** The processor adds the immediate value 3 to the register value 80. The resulting value is **83**.
5. **ORI Operation:** The processor performs a bitwise OR operation between the register value 2 and the immediate value 1. The resulting value is **3**.
6. **Load Word Address Calculation:** The processor calculates the effective address for the load instruction by adding the offset 15 to the value −22. The resulting address is **−7**. The actual loaded data depends on the data-memory contents and address-handling implementation.
7. **Program Counter Operation:** The Program Counter progresses through sequential instruction addresses, generally increasing by 4 bytes for each instruction.
8. **Instruction Decoding:** The opcode, rs1, rs2, and rd signals change according to the instruction fields, identifying the operation and the registers involved.
9. **Register File Operation:** The read_data1 and read_data2 signals show the values obtained from the source registers, while write_data carries the result selected for write-back.

8. ADVANTAGES AND DISADVANTAGES

8.1 Advantages

- **Simple Design:** The architecture is easy to design and implement because each instruction is executed within a single clock cycle without overlapping instruction stages.
- **Easy to Understand:** The processor has a straightforward datapath consisting of instruction memory, a register file, an ALU, data memory, and a control unit.
- **Simplified Control Unit:** Since instructions are not pipelined, there is no need for complex pipeline control or hazard detection mechanisms.
- **Predictable Execution:** Each instruction completes in one clock cycle,
- **Long Clock Period:** The clock period must be long enough to accommodate the slowest instruction, which limits the processor's maximum operating frequency.

8.2 Disadvantages

- **Low Performance:** The processor completes only one instruction per clock cycle, limiting its throughput compared with architectures that use pipelining.
- **Inefficient Hardware Utilization:** Some hardware components remain unused during certain instructions. For example, data memory is not required for ordinary arithmetic operations.
- **Slow Memory Operations:** Instructions involving data-memory access may require more time than simple arithmetic instructions, but all instructions must fit within the same clock period.

9. APPLICATIONS

- **Educational Purposes:** Used in academic projects to understand processor architecture, instruction execution, datapath design, and control-unit operation.
- **Embedded Systems:** Can be used as a basic processor design for simple embedded applications where computational requirements are limited.
- **Digital System Design:** Helps students and researchers understand the integration of instruction memory, register files, ALUs, data memory, and control units.
- **Processor Simulation and Verification:** Used to test and verify arithmetic, logical, memory-access, and branching instructions through Verilog simulation.
- **FPGA-Based Implementation:** Can be implemented on Field-Programmable Gate Arrays (FPGAs) to demonstrate the practical operation of a simple processor.

10. CONCLUSION

The software project on 32-bit Single-Cycle RISC-V Architecture without Pipelining provided a comprehensive understanding of processor architecture and instruction execution using Verilog HDL. The project involved designing and simulating the processor by implementing essential components, including instruction memory, a register file, an Arithmetic Logic Unit (ALU), a control unit, data memory, and a Program Counter (PC).

The simulation demonstrated the execution of various RISC-V instructions, including arithmetic, logical, immediate, memory-access, and conditional branch instructions. The generated simulation waveforms were used to observe the changes in internal signals, register values, instruction addresses, and computational results.

Through this project, the functionality of the processor was analyzed by examining instruction fetching, decoding, execution, and result write-back. The project also helped develop practical skills in Verilog programming, processor simulation, waveform analysis, and debugging.

Overall, this project establishes a foundation for understanding the design and simulation of RISC-V processors. The developed model can be further enhanced by adding support for more instructions, improving the processor's control logic, and implementing pipelining to increase instruction throughput.

processor organization and comparing single-cycle architectures with multicycle and pipelined designs.

11. RESULT

The software project on 32-bit Single-Cycle RISC-V Architecture without Pipelining was implemented and simulated using Verilog HDL. The processor model demonstrates the execution of arithmetic, logical, immediate, memory-access, and conditional branching instructions.

The simulation waveforms illustrate instruction fetching, decoding, register operations, ALU computations, Program Counter updates, and result write-back. The internal signals and computational results were analyzed to understand the flow of instructions through the processor.

The project successfully demonstrates the fundamental working principles of a single-cycle RISC-V processor and provides practical experience in hardware description language programming, processor simulation, and waveform analysis. It also establishes a foundation for further development and performance improvements in processor architecture.